Installation and Setup of ESP32 on VAMAN Board in Ubuntu 24.04

STEP 1 Install ESP-IDF

→ sudo apt update

Install dependencies

→ sudo apt install git wget flex bison gperf python3 python3-pip python3-venv \
cmake ninja-build ccache libffi-dev libssl-dev dfu-util libusb-1.0-0

Clone ESP-IDF

→ cd ~

→ git clone --recursive https://github.com/espressif/esp-idf.git

Install tools

→ cd ~/esp-idf

→ ./install.sh esp32

Load environment

→ source ~/esp-idf/export.sh

Verify

→ idf.py --version

Expected output:

ESP-IDF v6.x.x

Verify the compiler

→ xtensa-esp32-elf-gcc --version

You should see something like:

xtensa-esp-elf-gcc (Espressif ...)

Load the ESP-IDF environment

Each new terminal requires:

→ cd ~/esp-idf
→ source export.sh

You should see output similar to:

Checking Python compatibility...
Adding ESP-IDF tools to PATH...
Done! You can now compile ESP-IDF projects.

Verify esptool

→ python3 -m esptool version

or

→ esptool.py version

Expected:

esptool.py v5.x.x

Verify Python
→ python3 --version

You already have:

Python 3.12.3

which is fully supported.

Check USB-UART Detection

Connect your USB-to-TTL adapter and run:

→lsusb



```
r-309@r-309-ThinkCentre-neo-50s-Gen-3:~/esp-idf$ lsusb
Bus 001 Device 001: ID 1d6b:0002 Linux Foundation 2.0 root hub
Bus 001 Device 003: ID 17ef:6099 Lenovo Lenovo Traditional USB Keyboard
Bus 001 Device 004: ID 8087:0026 Intel Corp. AX201 Bluetooth
Bus 001 Device 009: ID 17ef:608d Lenovo Optical Mouse
Bus 001 Device 010: ID 10c4:ea60 Silicon Labs CP210x UART Bridge
Bus 002 Device 001: ID 1d6b:0003 Linux Foundation 3.0 root hub
r-309@r-309-ThinkCentre-neo-50s-Gen-3:~/esp-idf$
```

The device was detected as Silicon Labs CP210x UART Bridge

Check all serial devices

Instead of only checking ttyUSB*, run:

→ ls /dev/tty*

```
r-309@r-309-ThinkCentre-neo-50s-Gen-3:~/esp-idf$ ls /dev/tty*
/dev/tty     /dev/tty16  /dev/tty24  /dev/tty32  /dev/tty40  /dev/tty49  /dev/tty57  /dev/tty8       /dev/ttyS14  /dev/ttyS22  /dev/ttyS30
/dev/tty0    /dev/tty17  /dev/tty25  /dev/tty33  /dev/tty41  /dev/tty5   /dev/tty58  /dev/tty9       /dev/ttyS15  /dev/ttyS23  /dev/ttyS31
/dev/tty1    /dev/tty18  /dev/tty26  /dev/tty34  /dev/tty42  /dev/tty50  /dev/tty59  /dev/ttyprintk  /dev/ttyS16  /dev/ttyS24  /dev/ttyS4
/dev/tty10   /dev/tty19  /dev/tty27  /dev/tty35  /dev/tty43  /dev/tty51  /dev/tty6   /dev/ttyS0      /dev/ttyS17  /dev/ttyS25  /dev/ttyS5
/dev/tty11   /dev/tty2   /dev/tty28  /dev/tty36  /dev/tty44  /dev/tty52  /dev/tty60  /dev/ttyS1      /dev/ttyS18  /dev/ttyS26  /dev/ttyS6
/dev/tty12   /dev/tty20  /dev/tty29  /dev/tty37  /dev/tty45  /dev/tty53  /dev/tty61  /dev/ttyS10     /dev/ttyS19  /dev/ttyS27  /dev/ttyS7
/dev/tty13   /dev/tty21  /dev/tty3   /dev/tty38  /dev/tty46  /dev/tty54  /dev/tty62  /dev/ttyS11     /dev/ttyS2   /dev/ttyS28  /dev/ttyS8
/dev/tty14   /dev/tty22  /dev/tty30  /dev/tty39  /dev/tty47  /dev/tty55  /dev/tty63  /dev/ttyS12     /dev/ttyS20  /dev/ttyS29  /dev/ttyS9
/dev/tty15   /dev/tty23  /dev/tty31  /dev/tty4   /dev/tty48  /dev/tty56  /dev/tty7   /dev/ttyS13     /dev/ttyS21  /dev/ttyS3   /dev/ttyUSB0
```

The /dev/ttyUSB0 is identified

Monitor the kernel while plugging in the adapter

Open one terminal and run:

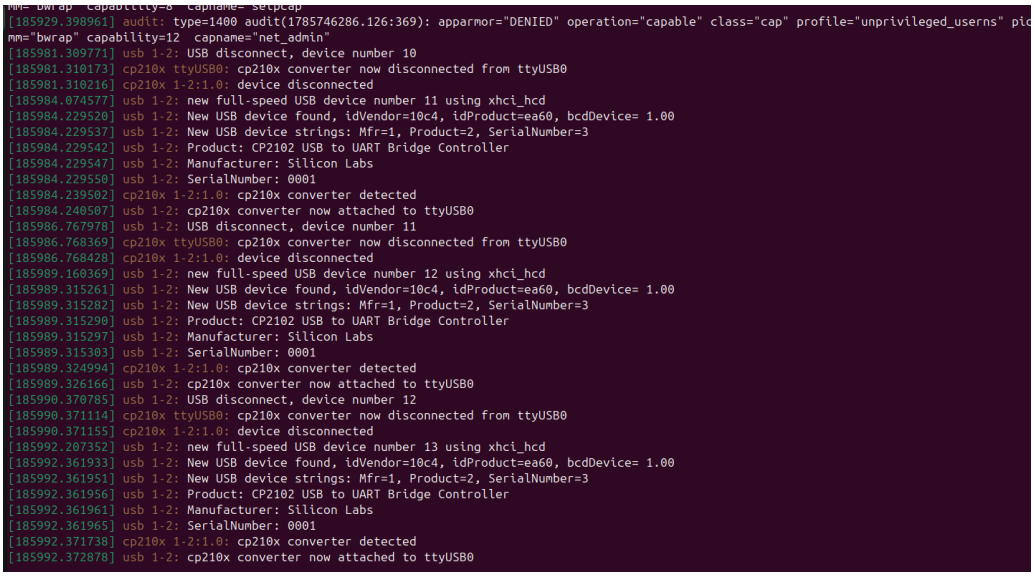
sudo dmesg -w

Leave it running.

Now:

1. Unplug the USB-to-TTL adapter.
2. Wait 5 seconds.
3. Plug it back in.

Copy the **new lines** that appear.



The cp210x is detecting by kernel

Go with the next commands

→lsmod | grep cp210x

→ls /dev/ttyUSB* /dev/ttyACM* 2>/dev/null

Which should detect the **/dev/ttyUSB0**

Now connect the UART USB TO VAMAN Board as follows

USB

  │

  ▼

CP2102 USB-to-TTL

  ├── TX → ESP32 RX

  ├── RX → ESP32 TX

  ├── GND → ESP32 GND

  ├── 3.3V → ESP32 3.3V

  └── **GPIO0 → GND** (only while flashing)

(press the USR buttorn to blink the green light which represents the flashing is started and then make sure the GPIO0 and GND are connected while flashing)

→ groups

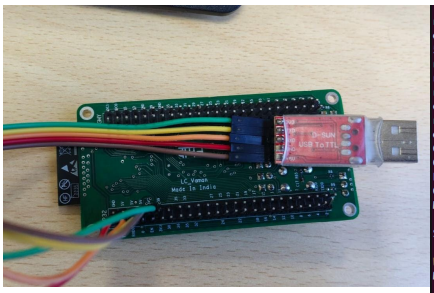
→ esptool version

→ls -l /dev/ttyUSB0

Verify Communication with the ESP32

→ esptool --chip esp32 -p /dev/ttyUSB0 chip-id

The outputs of the above commands were as follows



```
r-309@r-309-ThinkCentre-neo-50s-Gen-3:~/esp-idf$ esptool version
esptool v5.3.1
5.3.1
r-309@r-309-ThinkCentre-neo-50s-Gen-3:~/esp-idf$ groups
r-309 adm dialout cdrom sudo dip plugdev users lpadmin docker
r-309@r-309-ThinkCentre-neo-50s-Gen-3:~/esp-idf$ ls -l /dev/ttyUSB0
crw-rw---- 1 root dialout 188, 0 Aug  3 14:22 /dev/ttyUSB0
r-309@r-309-ThinkCentre-neo-50s-Gen-3:~/esp-idf$ esptool --chip esp32 -p /dev/ttyUSB0 chip-id
esptool v5.3.1
Connected to ESP32 on /dev/ttyUSB0:
Chip type:          ESP32-D0WD-V3 (revision v3.0)
Features:           Wi-Fi, BT, Dual Core + LP Core, 240MHz, Vref calibration in eFuse, Coding Scheme None
Crystal frequency:  40MHz
MAC:                c0:49:ef:bf:c7:d8

Stub flasher running.

Warning: ESP32 has no chip ID. Reading MAC address instead.
MAC:                c0:49:ef:bf:c7:d8

Hard resetting via RTS pin...
r-309@r-309-ThinkCentre-neo-50s-Gen-3:~/esp-idf$
```

Now we will create a new ESP-IDF project folder for the 7-segment display instead of modifying the example folder.

We will do this from scratch.

Assumption:

VAMAN board = ESP32
ESP-IDF = working
Display = common anode 7-segment
We will directly control the 7 segments using ESP32 GPIOs.
No 7447 IC initially (direct GPIO test).

1. Go to ESP-IDF projects folder

Create a workspace:

→ cd ~/esp-idf
→ mkdir VAMAN_PROJECTS
→ cd VAMAN_PROJECTS

2. Create new 7-segment project

Use ESP-IDF example template:

→ idf.py create-project seven_segment_vaman

Enter folder:

→cd seven_segment_vaman

Check files:

→ ls

You should see:

CMakeLists.txt
main

3. Check main folder

→ cd main

→ls

You will see:

seven_segment_vaman.c
CMakeLists.txt

(or main.c depending on ESP-IDF version)

4. Open the C file

Example:

→ nano seven_segment_vaman.c

Delete everything and add this code by using CTRL+K.

ESP32 Common Anode 7-Segment Code

Connections:

Segment	ESP32 GPIO
a	GPIO16
b	GPIO17
c	GPIO18
d	GPIO19
e	GPIO21
f	GPIO22
g	GPIO23

Common anode:

Common pin → 3.3V
Segment ON = LOW

Segment OFF = HIGH

```
#include <stdio.h>
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "driver/gpio.h"


#define SEG_A GPIO_NUM_16
#define SEG_B GPIO_NUM_17
#define SEG_C GPIO_NUM_18
#define SEG_D GPIO_NUM_19
#define SEG_E GPIO_NUM_21
#define SEG_F GPIO_NUM_22
#define SEG_G GPIO_NUM_23


int segments[7] = {
   SEG_A,
   SEG_B,
   SEG_C,
   SEG_D,
   SEG_E,
   SEG_F,
   SEG_G
};


/*
Common Anode:
0 = ON
1 = OFF
*/


int digits[10][7] =
{
   //a b c d e f g

   {0,0,0,0,0,0,1}, //0
   {1,0,0,1,1,1,1}, //1
   {0,0,1,0,0,1,0}, //2
   {0,0,0,0,1,1,0}, //3
```

```
    {1,0,0,1,1,0,0}, //4
    {0,1,0,0,1,0,0}, //5
    {0,1,0,0,0,0,0}, //6
    {0,0,0,1,1,1,1}, //7
    {0,0,0,0,0,0,0}, //8
    {0,0,0,0,1,0,0}  //9
};


void display_digit(int num)
{

    for(int i=0;i<7;i++)
    {
        gpio_set_level(
            segments[i],
            digits[num][i]
        );
    }

}


void app_main(void)
{

    for(int i=0;i<7;i++)
    {
        gpio_reset_pin(segments[i]);

        gpio_set_direction(
            segments[i],
            GPIO_MODE_OUTPUT
        );
    }


    while(1)
    {

        for(int i=0;i<10;i++)
        {

            display_digit(i);
```

```
            vTaskDelay(
               1000 / portTICK_PERIOD_MS
            );

        }

    }

}
```

Save:

CTRL + O
ENTER
CTRL + X

5. Configure ESP32 target

Go back:

→ cd ~/esp-idf/VAMAN_PROJECTS/seven_segment_vaman

Set target:

→ idf.py set-target esp32

6. Build

→ idf.py build

Successful message:
Project build complete.

7. Connect VAMAN

Check port:

→ ls /dev/ttyUSB*

Expected:

/dev/ttyUSB0

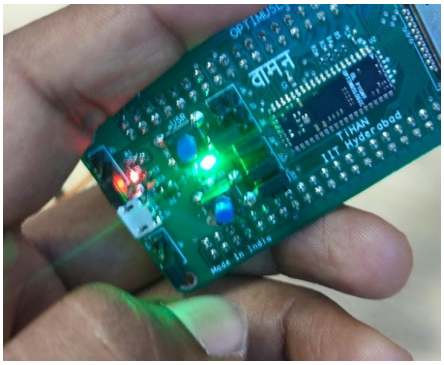
Verify Communication with the ESP32

→ esptool --chip esp32 -p /dev/ttyUSB0 chip-id

If error occurs once check if there are any loose connections and enter the virtual environment By running

→. ~/esp-idf/export.sh

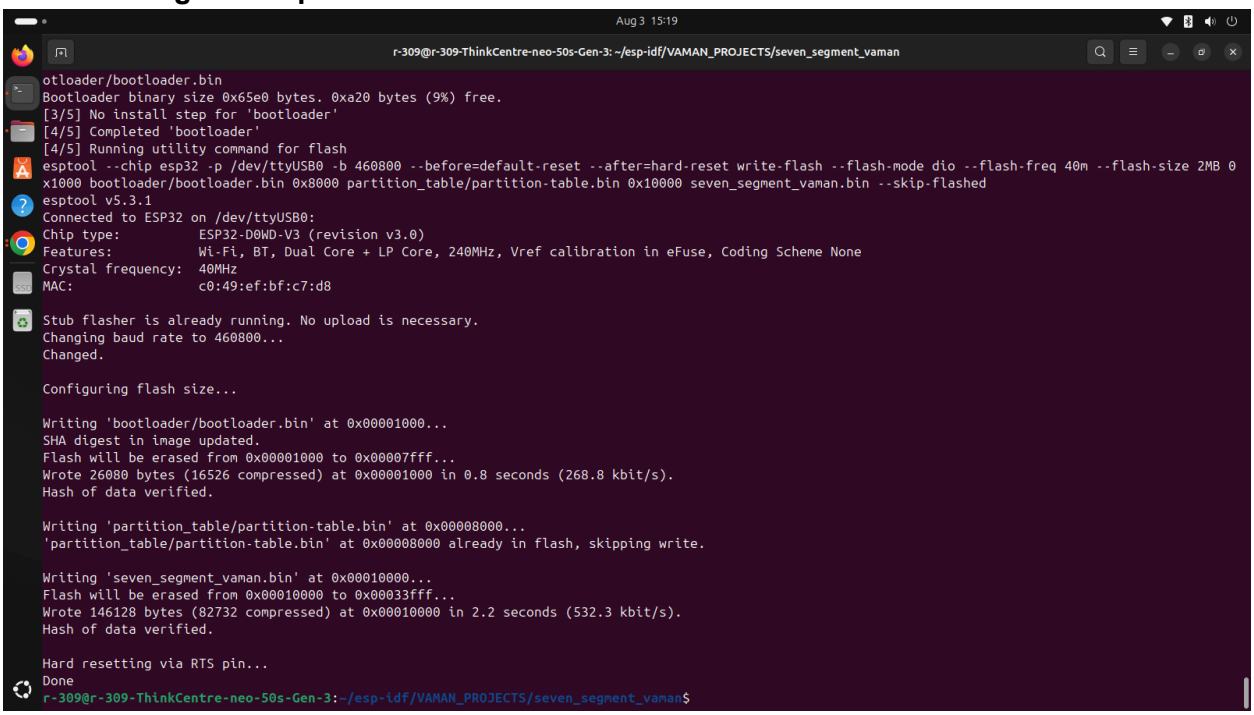
And then verify chip environment and make sure to keep GPI0 and GND in while flashing and green light should glow by pressing USR



8. Flash seven segment firmware

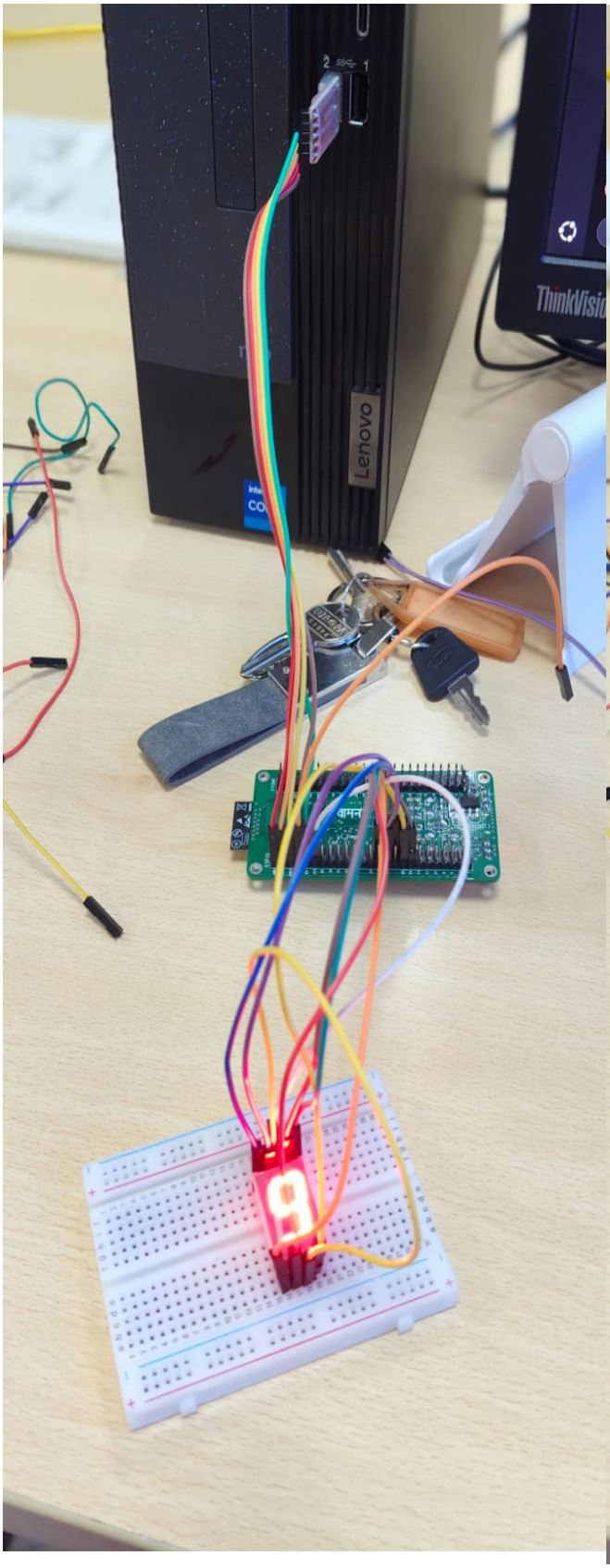
→ idf.py -p /dev/ttyUSB0 flash

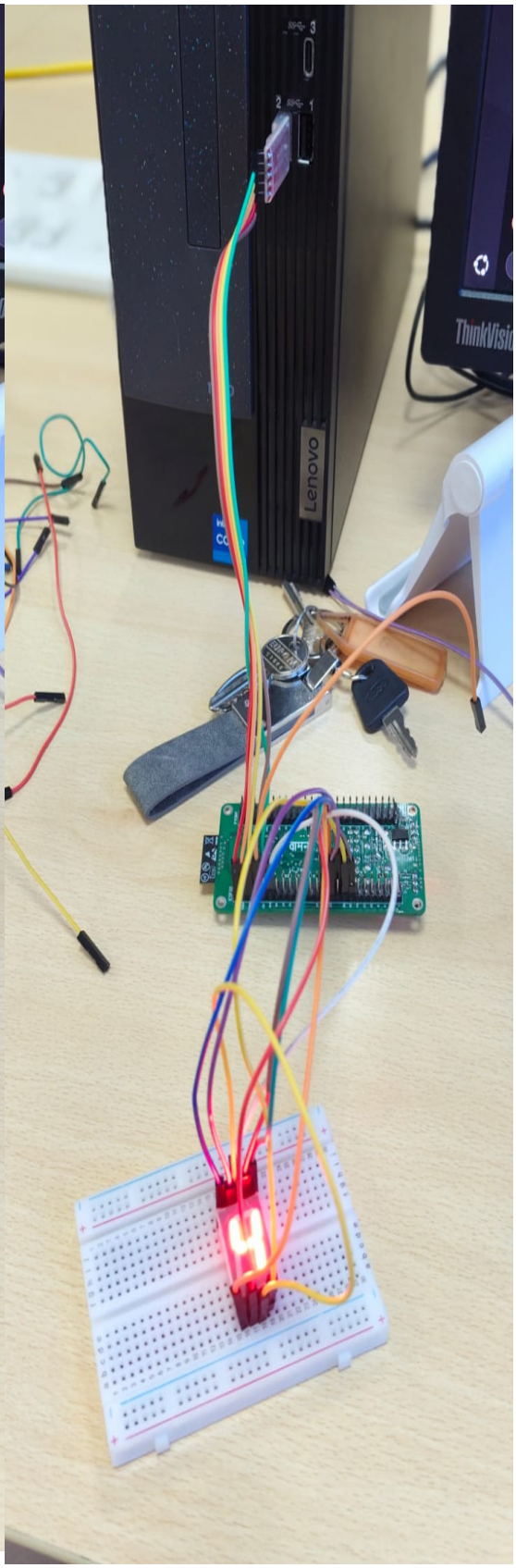
After flashing the output should be like as FOLLOWS



Press RESET.

The output of the current code is running of numbers 0→1→2→3→4→5→6→7→8→9→0


Lenovo
ThinkVisi
9


Lenovo
ThinkVisi
4